Material Behavior Prediction

Milestone 3 | Preprocessing Complete

Complete Technical Guide to Milestone 3: Data Preprocessing & Leakage Prevention

1. The Philosophy of Preprocessing in This Project

In machine learning, it is tempting to apply complex transformations blindly. Our philosophy is different:

> **The Principle of Parsimony**
>
> **Handle only what is strictly required by the data and problem.**
> Because our dataset has 0 missing values and 0 categorical text features, we do not need imputation or one-hot encoding. We only perform the essential steps: duplicate handling, train/test splitting, and anti-leakage scaling.

2. Detailed Breakdown of Milestone 3 Steps

1 Handling Duplicate Experimental Rows

The Decision: Dropped 25 exact duplicate rows (reducing dataset from 1,030 → 1,005 unique formulations).

Why Dropping Duplicates is Critical:

- In materials experiments, replicate test cylinders are cast to check test consistency.
- If an identical formulation appears in both the training set and the testing set, the model is tested on data it already memorized during training. This creates **Data Leakage** and artificially inflates evaluation metrics.
- By removing exact duplicates, we guarantee that the test set evaluates genuine generalization to unseen mixes.

2 Separating Features (X) & Target (y)

Inputs (X • 8 features): `cement, slag, ash, water, superplastic, coarseagg, fineagg, age`
Output (y • continuous target): `strength` (Compressive Strength in MPa)

3 Train/Test Split (80% / 20%)

We partition the 1,005 clean formulations into two strictly isolated subsets:

Dataset Partition	Number of Samples	Percentage	Mean Compressive Strength	Purpose
Training Set (X_{train}, y_{train})	**804**	80%	35.07 MPa	Used by ML algorithms to learn relationships and tune internal weights.
Testing Set (X_{test}, y_{test})	**201**	20%	35.98 MPa	Locked away; used strictly for final unbiased model evaluation.

Reproducibility with `random_state=42`: Using a fixed random seed guarantees that anyone running our code gets the exact same train/test split, making research results verifiable.

4 Feature Scaling (StandardScaler) & The Anti-Leakage Rule

Our 8 features have vastly different numeric magnitudes:

- Aggregate weights exceed $1,000 ext{ kg/m}^3$.
- Superplasticizer is only $0 ext{ to }32 ext{ kg/m}^3$.
- Curing age ranges from $1 ext{ to }365 ext{ days}$.

Without scaling, algorithms that compute geometric distances or linear gradients (such as Linear Regression, Ridge, Lasso, and KNN) give unfair weight to features with larger numbers.

Standardization Formula: $z = (x - \mu) / \sigma$ **(Transforms data to Mean = 0, Std = 1)**

> **The Golden Rule of Data Leakage Prevention**
>
> **1. Fit ONLY on Training Data:** `scaler.fit_transform(X_train)` computes the mean (μ) and standard deviation (σ) strictly from the 804 training samples.
> **2. Transform Testing Data:** `scaler.transform(X_test)` applies the *training* mean and standard deviation to the test set.
> **NEVER CALL `fit` or `fit_transform` on `X_test`!** Doing so would allow test set properties to leak into the pipeline before testing.

5 Reusable Module Architecture & Model Persistence

In `src/preprocessing.py`, we created:

- A standalone executable pipeline (`python src/preprocessing.py`).
- An importable function `prepare_data()` that can be called directly by future training and evaluation scripts without copying and pasting code.
- Fitted scaler persistence: saved to `models/scaler.joblib` so that new unseen material formulations can be transformed identically during future inference (Milestone 8).

3. Summary Checklist Before Milestone 4

- [x] 25 exact duplicate rows removed (1,005 unique samples retained).
- [x] Unbiased 80/20 train/test split created with balanced target means (35.07 vs 35.98 MPa).
- [x] Zero data leakage verified (StandardScaler learned parameters solely from `X_train`).
- [x] Both unscaled and scaled feature matrices prepared (unscaled for Decision Trees / Random Forests; scaled for Linear Regression).
- [x] Scaler serialized to `models/scaler.joblib` .